

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Muhammad Kurniawan Pasya

NIM. 2510817210026

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree, ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya

NIM : 2510817210026

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	5
DAFTAR TABEL	6
SOAL 1.....	7
A. Source Code.....	9
B. Output Program.....	11
C. Pembahasan	12
SOAL 2.....	14
A. Source Code.....	16
B. Output Program.....	17
C. Pembahasan	18
SOAL 3.....	20
A. Source Code.....	22
B. Output Program.....	25
C. Pembahasan	25
SOAL 4.....	28
A. Source Code.....	30
B. Output Program.....	33
C. Pembahasan	33
SOAL 5.....	36
A. Source Code.....	38
B. Output Program.....	42

C. Pembahasan	43
TAUTAN GIT	47

DAFTAR GAMBAR

Gambar 1 Screenshot Output Progam Soal 1	11
Gambar 2 Screenshot Output Program Soal 2.....	17
Gambar 3 Screenshot Output Program Soal 3.....	25
Gambar 4 Screenshot Output Program Soal 4.....	33
Gambar 5 Screenshot Output Program Soal 5.....	42

DAFTAR TABEL

Tabel 1 Source Code prob1.cpp	9
Tabel 2 Source Code prob2.cpp	16
Tabel 3 Source Code prob3.cpp	22
Tabel 4 Source Code prob4.cpp	30
Tabel 5 Source Code prob5.cpp	38

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.

- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

A. Source Code

Tabel 1 Source Code probl.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <map></code>
4	<code>using namespace std;</code>
5	
6	<code>int main() {</code>
7	<code> int N;</code>
8	<code> cin >> N;</code>
9	
10	<code> vector<char> vertex(N);</code>
11	<code> map<char, int> pos;</code>
12	
13	<code> for (int i = 0; i < N; i++) {</code>
14	<code> cin >> vertex[i];</code>
15	<code> pos[vertex[i]] = i;</code>
16	<code> }</code>
17	
18	<code> vector<vector<int>> matrix(N, vector<int>(N, 0));</code>
19	

```
20     int M;
21     cin >> M;
22
23     for (int i = 0; i < M; i++) {
24         char u, v;
25         int w;
26
27         cin >> u >> v >> w;
28
29         matrix[pos[u]][pos[v]] = w;
30     }
31
32     cout << "Adjacency Matrix:\n";
33
34     cout << " ";
35     for (char c : vertex) {
36         cout << " " << c;
37     }
38     cout << '\n';
39
40     for (int i = 0; i < N; i++) {
41         cout << vertex[i];
42
43         for (int j = 0; j < N; j++) {
44             cout << " " << matrix[i][j];
45         }
46
47         cout << '\n';
48     }
49
50     return 0;
```

B. Output Program

```
PS D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PR  
ree-RyuuZev> g++ prob1.cpp -o prob1; ./prob1  
5  
A B C D E  
6  
A B 4  
A C 2  
B D 7  
C B 1  
C E 6  
D C 3  
Adjacency Matrix:  
  A B C D E  
A 0 4 2 0 0  
B 0 0 0 7 0  
C 0 1 0 0 6  
D 0 0 3 0 0  
E 0 0 0 0 0
```

Gambar 1 Screenshot Output Program Soal 1

C. Pembahasan

Jadi kode ini merupakan implementasi Adjacency Matrix untuk merepresentasikan sebuah graph berbobot dan berarah. Input yang diberikan berupa jumlah vertex, nama vertex, jumlah edge, serta bobot masing masing edge. Output yang dihasilkan adalah adjacency matrix yang menunjukkan hubungan antar vertex beserta bobot edgenya.

Program ini menggunakan dua struktur data utama yaitu `vector` dan `map`. `vector` digunakan untuk menyimpan nama vertex dan adjacency matrix, sedangkan `map` digunakan untuk menyimpan pasangan nama vertex dengan indeksnya pada matriks.

Pertama, deklarasi `vector<char> vertex(N)` dan `map<char, int> pos`.

Vector `vertex` digunakan untuk menyimpan seluruh nama vertex yang diinput pengguna. Sedangkan `pos` digunakan untuk menyimpan posisi setiap vertex pada matriks. Misalnya jika vertex yang dimasukkan adalah A, B, C, D, dan E maka map akan menyimpan $A \rightarrow 0$, $B \rightarrow 1$, $C \rightarrow 2$, $D \rightarrow 3$, dan $E \rightarrow 4$. Dengan cara ini program dapat langsung mengetahui indeks suatu vertex tanpa perlu melakukan pencarian berulang. Proses pengisian vertex dilakukan sebanyak N kali sehingga memiliki time complexity $O(N)$. Space complexitynya $O(N)$ karena menyimpan N buah vertex dan N pasangan data pada map.

Lalu deklarasi `vector<vector<int>> matrix(N, vector<int>(N, 0))` digunakan untuk membuat adjacency matrix berukuran $N \times N$. Seluruh elemen matriks diinisialisasi dengan nilai 0 yang menunjukkan bahwa belum terdapat hubungan antar vertex. Karena matriks yang dibuat berukuran $N \times N$, maka time complexity proses inisialisasi adalah $O(N^2)$. Space complexitynya juga $O(N^2)$ karena program harus menyediakan memori untuk seluruh elemen matriks.

Proses input edge dilakukan menggunakan perulangan sebanyak M kali. Pada setiap iterasi program membaca vertex asal, vertex tujuan, dan bobot edge. Setelah itu program mencari indeks kedua vertex melalui map dan langsung menyimpan bobot edge ke posisi matriks yang sesuai.

Sebagai contoh jika dimasukkan edge A B 4, maka nilai 4 akan disimpan pada posisi matriks yang merepresentasikan hubungan dari A ke B. Karena proses pengisian hanya berupa akses langsung ke matriks, maka setiap operasi memiliki time complexity $O(1)$. Namun karena

dilakukan sebanyak M kali, total time complexity bagian ini menjadi $O(M)$. Space complexity tambahan yang digunakan adalah $O(1)$ karena hanya memakai beberapa variabel sementara.

Setelah itu, proses pencetakan adjacency matrix dilakukan dengan dua perulangan bersarang. Program terlebih dahulu mencetak seluruh nama vertex sebagai header kolom. Setelah itu setiap baris matriks dicetak bersama nama vertex yang mewakili baris tersebut. Karena seluruh elemen matriks harus ditampilkan satu per satu, proses ini memiliki time complexity $O(N^2)$. Space complexity-nya $O(1)$ karena tidak membuat struktur data tambahan.

Terakhir, fungsi `main` bertugas mengatur seluruh alur program. Program dimulai dengan membaca jumlah vertex, menyimpan nama vertex, membuat adjacency matrix, membaca seluruh edge, mengisi bobot edge ke dalam matriks, lalu mencetak hasil akhirnya.

Secara keseluruhan kompleksitas waktu program adalah $O(N + M + N^2)$. Komponen $O(N)$ berasal dari input vertex, $O(M)$ berasal dari input dan pengisian edge, sedangkan $O(N^2)$ berasal dari proses pencetakan adjacency matrix. Karena $O(N^2)$ merupakan komponen terbesar, maka kompleksitas waktu program secara dominan dapat dituliskan sebagai $O(N^2)$. Untuk kompleksitas ruang, kebutuhan memori terbesar berasal dari adjacency matrix berukuran $N \times N$ sehingga space complexity program adalah $O(N^2)$.

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik. Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk

A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2 Source Code prob2.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int N;
7      cin >> N;
8
9      vector<char> vertex(N);
10
11     for (int i = 0; i < N; i++) {
12         cin >> vertex[i];
13     }
14
15     vector<vector<int>> matrix(N, vector<int>(N));
16
17     for (int i = 0; i < N; i++) {
18         for (int j = 0; j < N; j++) {
19             cin >> matrix[i][j];
20         }
21     }
22
23     cout << "Adjacency List:\n";
24
25     for (int i = 0; i < N; i++) {
26         cout << vertex[i] << " -> ";
27
28         bool first = true;
29
30         for (int j = 0; j < N; j++) {
31             if (matrix[i][j] > 0) {
32                 if (!first) cout << ", ";
33
```

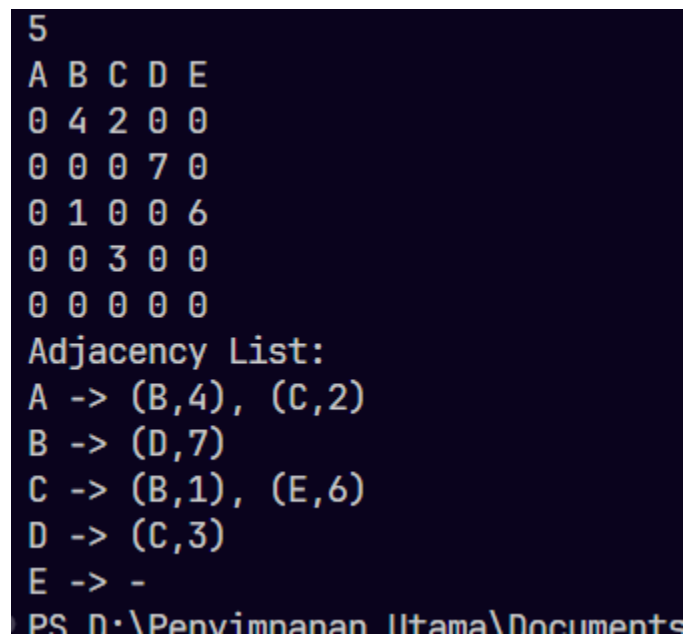


```

34         cout << "(" << vertex[j]
35             << "," << matrix[i][j] << ")";
36
37         first = false;
38     }
39 }
40
41 if (first) cout << "-";
42
43 cout << '\n';
44 }
45
46 return 0;
47 }

```

B. Output Program



```

5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS D:\Penyimpanan Utama\Documents

```

Gambar 2 Screenshot Output Program Soal 2

C. Pembahasan

Implementasi algoritma untuk mengubah Adjacency Matrix menjadi Adjacency List pada graph berbobot dan berarah. Input yang diberikan berupa jumlah vertex, nama setiap vertex, serta adjacency matrix yang berisi bobot hubungan antar vertex. Output yang dihasilkan adalah adjacency list yang menampilkan vertex tujuan beserta bobot edge yang terhubung.

Pada program ini digunakan dua buah vector. Vector pertama digunakan untuk menyimpan nama vertex, sedangkan vector dua dimensi digunakan untuk menyimpan adjacency matrix yang diinput pengguna.

Pertama, program membaca jumlah vertex yang akan digunakan pada graph. Nilai tersebut disimpan pada variabel `N`. Setelah itu dibuat `vector<char> vertex(N)` untuk menyimpan nama setiap vertex. Proses input nama vertex dilakukan sebanyak `N` kali menggunakan perulangan. Karena dilakukan sebanyak `N` kali, maka time complexity pada bagian ini adalah $O(N)$. Space complexity nya juga $O(N)$ karena menyimpan `N` buah nama vertex.

Terus program membuat adjacency matrix menggunakan `vector<vector<int>> matrix(N, vector<int>(N))`. Matriks ini digunakan untuk menyimpan seluruh hubungan antar vertex beserta bobot edge nya.

Setelah matriks dibuat, program membaca seluruh elemen matriks menggunakan dua buah perulangan bersarang. Perulangan luar berjalan sebanyak `N` kali dan perulangan dalam juga berjalan sebanyak `N` kali sehingga total operasi yang dilakukan adalah $N \times N$. Oleh karena itu time complexity proses input matriks adalah $O(N^2)$. Space complexity nya juga $O(N^2)$ karena seluruh elemen matriks harus disimpan di memori.

Program mencetak adjacency list berdasarkan data yang terdapat pada adjacency matrix. Untuk setiap vertex, program melakukan penelusuran terhadap seluruh kolom pada baris yang bersesuaian. Jika ditemukan nilai lebih besar dari 0, berarti terdapat edge dari vertex tersebut menuju vertex lain sehingga pasangan vertex tujuan dan bobot edge dicetak ke layar.

Variabel `first` digunakan untuk mengatur tampilan tanda koma agar format output tetap rapi. Jika setelah seluruh kolom diperiksa tidak ditemukan edge, maka program mencetak tanda "-" untuk menunjukkan bahwa vertex tersebut tidak memiliki hubungan ke vertex lain.

Proses konversi dari adjacency matrix menjadi adjacency list dilakukan menggunakan dua perulangan bersarang yang masing masing berjalan sebanyak N kali. Karena seluruh elemen matriks harus diperiksa satu per satu, maka time complexity bagian ini adalah $O(N^2)$. Space complexity tambahan yang digunakan adalah $O(1)$ karena hanya menggunakan beberapa variabel sederhana seperti `first`, `i`, dan `j`.

Di fungsi utama yaitu fungsi `main` mengatur seluruh alur program mulai dari membaca jumlah vertex, membaca nama vertex, membaca adjacency matrix, melakukan konversi ke adjacency list, hingga menampilkan hasil akhirnya. Secara keseluruhan kompleksitas waktu program adalah $O(N + N^2 + N^2)$. Komponen $O(N)$ berasal dari input nama vertex, sedangkan dua komponen $O(N^2)$ berasal dari input adjacency matrix dan proses konversi sekaligus pencetakan adjacency list.

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$SR\ SC$

$FR\ FC$

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.

- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3 Source Code prob3.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <queue></code>
4	<code>using namespace std;</code>
5	
6	<code>struct State {</code>
7	<code> int r;</code>
8	<code> int c;</code>
9	<code>};</code>
10	
11	<code>int main() {</code>
12	<code> int R, C;</code>
13	<code> cin >> R >> C;</code>

```
14
15     vector<vector<int>> grid(R, vector<int>(C));
16
17     for (int i = 0; i < R; i++) {
18         for (int j = 0; j < C; j++) {
19             cin >> grid[i][j];
20         }
21     }
22
23     int sr, sc;
24     int fr, fc;
25
26     cin >> sr >> sc;
27     cin >> fr >> fc;
28
29     vector<vector<int>> dist(R, vector<int>(C, -1));
30
31     queue<State> q;
32
33     q.push({sr, sc});
34     dist[sr][sc] = 0;
35
36     int dr[4] = {-1, 1, 0, 0};
37     int dc[4] = {0, 0, -1, 1};
38
39     while (!q.empty()) {
40         State cur = q.front();
41         q.pop();
42
43         for (int k = 0; k < 4; k++) {
44             int nr = cur.r + dr[k];
```

```
45         int nc = cur.c + dc[k];
46
47         if (nr < 0 || nr >= R ||
48             nc < 0 || nc >= C)
49             continue;
50
51         if (grid[nr][nc] == 1)
52             continue;
53
54         if (dist[nr][nc] != -1)
55             continue;
56
57         dist[nr][nc] = dist[cur.r][cur.c] + 1;
58         q.push({nr, nc});
59     }
60 }
61
62 cout << dist[fr][fc] << '\n';
63
64 return 0;
65 }
```


B. Output Program

```
PS D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\Pr  
e-6-graph-and-tree-RyuuZev> g++ prob3.cpp -o prob3 ; ./prob3  
8 10  
0 0 0 0 1 0 0 0 0 0  
1 1 1 0 1 0 1 1 1 0  
0 0 1 0 0 0 0 0 1 0  
0 1 1 1 1 1 1 0 1 0  
0 0 0 0 0 0 1 0 0 0  
1 1 1 1 1 0 1 1 1 0  
0 0 0 0 1 0 0 0 1 0  
0 1 1 0 0 0 1 0 0 0  
0 0  
7 9  
16
```

Gambar 3 Screenshot Output Program Soal 3

C. Pembahasan

Kode ini adalah implementasi algoritma Breadth First Search (BFS) untuk mencari jarak terpendek dari suatu titik awal menuju titik tujuan pada sebuah grid. Input yang diberikan berupa jumlah baris dan kolom, isi grid yang terdiri dari nilai 0 dan 1, koordinat titik awal, serta koordinat titik tujuan. Nilai 0 menunjukkan sel yang dapat dilewati, sedangkan nilai 1 menunjukkan penghalang yang tidak dapat dilalui. Output yang dihasilkan berupa panjang lintasan terpendek dari titik awal ke titik tujuan.

Pada program ini terdapat sebuah struct bernama `State` yang berisi dua variabel yaitu `r` dan `c`. Variabel tersebut digunakan untuk menyimpan posisi baris dan kolom suatu sel pada grid. Struct ini digunakan agar koordinat dapat disimpan dan diproses dengan lebih mudah saat dimasukkan ke dalam queue.

Pertama, program membaca jumlah baris dan kolom yang disimpan pada variabel R dan C . Setelah itu dibuat `vector<vector<int>>` `grid` berukuran $R \times C$ untuk menyimpan isi grid yang diinput pengguna. Proses pembacaan grid dilakukan menggunakan dua perulangan bersarang sehingga seluruh sel dapat terisi.

Karena setiap elemen grid dibaca satu kali, maka time complexity bagian ini adalah $O(R \times C)$. Space complexity nya juga $O(R \times C)$ karena seluruh data grid disimpan dalam memori.

Program membaca koordinat titik awal yang disimpan pada variabel sr dan sc , serta koordinat titik tujuan yang disimpan pada variabel fr dan fc . Setelah itu dibuat matriks `dist` berukuran $R \times C$ yang diisi dengan nilai -1.

Matriks ini digunakan untuk menyimpan jarak dari titik awal menuju setiap sel yang berhasil dikunjungi. Nilai -1 menunjukkan bahwa sel tersebut belum pernah dikunjungi. Pembuatan matriks ini membutuhkan time complexity $O(R \times C)$ dan space complexity $O(R \times C)$.

Lalu ada sebuah `queue<State>` yang digunakan sebagai struktur utama pada algoritma BFS. Titik awal dimasukkan ke dalam queue dan nilai jaraknya pada matriks `dist` diisi 0 karena merupakan titik awal pencarian.

Selanjutnya dibuat dua array yaitu dr dan dc . Kedua array ini digunakan untuk menentukan empat arah pergerakan yang diperbolehkan, yaitu atas, bawah, kiri, dan kanan. Dengan cara ini program dapat memeriksa seluruh tetangga dari suatu posisi tanpa harus menuliskan logika pergerakan secara berulang.

Proses BFS dilakukan selama queue tidak kosong. Pada setiap iterasi, program mengambil posisi paling depan dari queue kemudian memeriksa empat sel tetangganya. Jika posisi baru berada di luar batas grid, maka posisi tersebut diabaikan. Jika sel tersebut merupakan penghalang dengan nilai 1, maka sel tersebut juga diabaikan. Selain itu, jika sel tersebut sudah pernah dikunjungi sebelumnya, maka tidak akan diproses kembali.

Apabila seluruh syarat terpenuhi, maka jarak pada sel baru diisi dengan nilai jarak sel saat ini ditambah satu. Setelah itu posisi baru dimasukkan ke dalam queue agar dapat diproses pada iterasi berikutnya. Karena setiap sel paling banyak dikunjungi satu kali dan setiap kunjungan hanya memeriksa empat arah, maka kompleksitas waktu proses BFS adalah $O(R \times C)$.

Terakhir, setelah seluruh proses BFS selesai dilakukan, program mencetak nilai pada `dist[fr][fc]` yang merupakan jarak terpendek dari titik awal menuju titik tujuan. Jika nilai tersebut tetap -1, berarti titik tujuan tidak dapat dicapai dari titik awal.

Secara keseluruhan, kompleksitas waktu program berasal dari pembacaan grid $O(R \times C)$, inisialisasi matriks jarak $O(R \times C)$, dan proses BFS $O(R \times C)$. Oleh karena itu total kompleksitas waktunya adalah: **Time Complexity = $O(R \times C)$**

Untuk kompleksitas ruang, program menggunakan grid berukuran $R \times C$, matriks jarak berukuran $R \times C$, serta queue yang dalam kondisi terburuk dapat menyimpan hingga $R \times C$ posisi. Oleh karena itu kompleksitas ruang program adalah: **Space Complexity = $O(R \times C)$**

Dari data uji yang digunakan terdapat grid berukuran 8×10 dengan titik awal berada pada koordinat (0,0) dan titik tujuan berada pada koordinat (7,9). Program melakukan penelusuran menggunakan BFS dan menghasilkan jarak terpendek sebesar 16 langkah.

Hasil ini menunjukkan kalo algoritma BFS berhasil menemukan lintasan terpendek karena BFS selalu mengeksplorasi simpul berdasarkan urutan jarak terdekat terlebih dahulu sebelum berpindah ke jarak yang lebih jauh.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya.

Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$SR\ SC$

$FR\ FC$

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.

- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4 Source Code prob4.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>using namespace std;</code>
4	
5	<code>int R, C;</code>
6	<code>int sr, sc;</code>
7	<code>int fr, fc;</code>
8	
9	<code>vector<vector<int>> grid;</code>
10	<code>vector<vector<bool>> visited;</code>
11	
12	<code>long long totalPath = 0;</code>
13	

```
14 int dr[4] = {-1, 1, 0, 0};
15 int dc[4] = {0, 0, -1, 1};
16
17 void dfs(int r, int c) {
18     if (r == fr && c == fc) {
19         totalPath++;
20         return;
21     }
22
23     visited[r][c] = true;
24
25     for (int k = 0; k < 4; k++) {
26         int nr = r + dr[k];
27         int nc = c + dc[k];
28
29         if (nr < 0 || nr >= R ||
30             nc < 0 || nc >= C)
31             continue;
32
33         if (grid[nr][nc] == 1)
34             continue;
35
36         if (visited[nr][nc])
37             continue;
38
39         dfs(nr, nc);
40     }
41
42     visited[r][c] = false;
43 }
44
```

```
45 int main() {
46     cin >> R >> C;
47
48     grid.assign(R, vector<int>(C));
49
50     for (int i = 0; i < R; i++) {
51         for (int j = 0; j < C; j++) {
52             cin >> grid[i][j];
53         }
54     }
55
56     cin >> sr >> sc;
57     cin >> fr >> fc;
58
59     visited.assign(R, vector<bool>(C, false));
60
61     dfs(sr, sc);
62
63     cout << "\nOutput" << endl;
64     cout << totalPath << '\n';
65
66     return 0;
67 }
```


B. Output Program

```
PS D:\Penyimpanan Utama\Documents\#KULIAH
Zev> g++ prob4.cpp -o prob4 ; ./prob4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5

Output
4
```

Gambar 4 Screenshot Output Program Soal 4

C. Pembahasan

Implementasi algoritma Depth First Search (DFS) untuk menghitung jumlah seluruh jalur yang dapat ditempuh dari titik awal menuju titik tujuan pada sebuah grid.

Input yang diberikan berupa jumlah baris dan kolom, isi grid yang terdiri dari nilai 0 dan 1, koordinat titik awal, serta koordinat titik tujuan. Nilai 0 menunjukkan sel yang dapat dilewati, sedangkan nilai 1 menunjukkan penghalang yang tidak dapat dilalui. Output yang dihasilkan adalah jumlah seluruh jalur yang mungkin dari titik awal menuju titik tujuan.

Di kode ini terdapat beberapa variabel global yaitu `R`, `C`, `sr`, `sc`, `fr`, `fc`, `grid`, `visited`, dan `totalPath`. Penggunaan variabel global bertujuan agar seluruh data dapat diakses langsung oleh fungsi DFS tanpa perlu dikirim sebagai parameter setiap kali fungsi dipanggil secara rekursif.

Pertama tama program membaca jumlah baris dan kolom yang disimpan pada variabel `R` dan `C`. Setelah itu dibuat `grid` berukuran $R \times C$ untuk menyimpan data labirin yang diinput pengguna. Proses input grid dilakukan menggunakan dua perulangan bersarang sehingga seluruh sel dapat terisi. Karena seluruh elemen grid dibaca satu kali, maka time complexity bagian ini adalah $O(R \times C)$. Space complexity nya juga $O(R \times C)$ karena seluruh data grid disimpan dalam memori.

Kemudian, program membaca koordinat titik awal dan titik tujuan. Setelah itu dibuat matriks `visited` berukuran $R \times C$ yang seluruh elemennya diisi dengan nilai `false`. Matriks ini digunakan untuk menandai apakah suatu sel sedang berada pada jalur yang sedang dieksplorasi atau belum. Dengan adanya matriks ini, program dapat menghindari perulangan tak berhingga akibat mengunjungi sel yang sama secara terus menerus.

Program memanggil fungsi `dfs(sr, sc)` untuk memulai pencarian dari titik awal. Pada fungsi DFS, langkah pertama yang dilakukan adalah memeriksa apakah posisi saat ini sama dengan posisi tujuan. Jika sama, maka variabel `totalPath` ditambah satu karena ditemukan satu jalur yang valid dari titik awal ke titik tujuan. Setelah itu fungsi langsung dihentikan menggunakan `return`.

Jika posisi saat ini belum mencapai tujuan, maka sel tersebut ditandai sebagai telah dikunjungi dengan mengubah nilai `visited[r][c]` menjadi `true`. Selanjutnya program memeriksa empat arah pergerakan yaitu atas, bawah, kiri, dan kanan menggunakan array `dr` dan `dc`.

Untuk setiap arah, program melakukan beberapa pengecekan. Pertama memastikan posisi baru masih berada di dalam batas grid. Kedua memastikan sel tersebut bukan penghalang yang bernilai 1. Ketiga memastikan sel tersebut belum pernah dikunjungi pada jalur yang sedang diproses.

Jika seluruh syarat terpenuhi, maka fungsi DFS dipanggil kembali secara rekursif pada posisi tersebut.

Setelah seluruh kemungkinan arah selesai diperiksa, program mengubah kembali nilai `visited[r][c]` menjadi `false`. Proses ini merupakan mekanisme [backtracking](#), yaitu mengembalikan keadaan ke kondisi sebelumnya sehingga sel tersebut dapat dieksplorasi kembali pada jalur pencarian yang berbeda.

Setelah proses DFS selesai dilakukan, program mencetak nilai `totalPath` yang berisi jumlah seluruh jalur yang berhasil ditemukan dari titik awal menuju titik tujuan.

Untuk kompleksitas waktu, kasus terburuk terjadi ketika hampir seluruh sel dapat dilewati dan DFS harus mencoba semua kemungkinan jalur yang ada. Karena jumlah kemungkinan jalur dapat bertambah sangat cepat seiring bertambahnya ukuran grid, maka kompleksitas waktu pada kasus terburuk bersifat eksponensial dan dapat dituliskan sebagai:

$$\text{Time Complexity} = O(4^{(R \times C)})$$

Hal ini terjadi karena dari setiap posisi DFS dapat mencoba hingga 4 kemungkinan arah pergerakan dan proses tersebut berlangsung secara rekursif hingga banyak tingkat.

Untuk kompleksitas ruang, program menggunakan grid berukuran $R \times C$, matriks `visited` berukuran $R \times C$, serta stack rekursi yang pada kasus terburuk dapat menyimpan hingga $R \times C$ pemanggilan fungsi. Oleh karena itu kompleksitas ruang program adalah:

$$\text{Space Complexity} = O(R \times C)$$

Pada data uji yang digunakan terdapat grid berukuran 5×6 dengan titik awal berada pada koordinat (0,0) dan titik tujuan berada pada koordinat (4,5).

Program melakukan penelusuran menggunakan DFS dan backtracking untuk mencoba seluruh kemungkinan jalur yang valid. Hasil yang diperoleh terdapat 4 jalur berbeda yang dapat digunakan untuk mencapai tujuan. Ini berarti algoritma DFS dengan backtracking berhasil mengeksplorasi seluruh kemungkinan lintasan dan menghitung jumlah jalur secara tepat tanpa menghitung jalur yang sama lebih dari satu kali.

SOAL 5

Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.

- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu

Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTtree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTtree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTtree yang dibentuk sudah konsisten dengan aturan RBTtree

A. Source Code

Tabel 5 Source Code prob5.cpp

1	#include <iostream>
2	#include <vector>
3	#include <string>
4	#include "RedBlackTree.h"
5	
6	using namespace std;
7	
8	void preorder(
9	const RedBlackTree::Node* node,
10	const RedBlackTree::Node* nil,
11	vector<int>& result) {
12	
13	if (node == nil node->isNil)
14	return;
15	
16	result.push_back(node->key);
17	
18	preorder(node->left, nil, result);
19	preorder(node->right, nil, result);

```
20 }
21
22 void inorder(
23     const RedBlackTree::Node* node,
24     const RedBlackTree::Node* nil,
25     vector<int>& result) {
26
27     if (node == nil || node->isNil)
28         return;
29
30     inorder(node->left, nil, result);
31
32     result.push_back(node->key);
33
34     inorder(node->right, nil, result);
35 }
36
37 void postorder(
38     const RedBlackTree::Node* node,
39     const RedBlackTree::Node* nil,
40     vector<int>& result) {
41
42     if (node == nil || node->isNil)
43         return;
44
45     postorder(node->left, nil, result);
46     postorder(node->right, nil, result);
47
48     result.push_back(node->key);
49 }
50
```

```
51 void printTraversal(  
52     const string& title,  
53     const vector<int>& data) {  
54  
55     cout << "[" << title << "]" :";  
56  
57     for (int x : data) {  
58         cout << " " << x;  
59     }  
60  
61     cout << '\n';  
62 }  
63  
64 int main() {  
65     int N;  
66     cin >> N;  
67  
68     RedBlackTree tree;  
69  
70     for (int i = 0; i < N; i++) {  
71         int x;  
72         cin >> x;  
73  
74         if (!tree.contains(x))  
75             tree.insert(x);  
76     }  
77  
78     int Q;  
79     cin >> Q;  
80  
81     if (tree.empty()) {
```



```
82         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
83         return 0;
84     }
85
86     while (Q--) {
87         string query;
88         cin >> query;
89
90         vector<int> pre;
91         vector<int> in;
92         vector<int> post;
93
94         if (query == "PREORDER") {
95             preorder(tree.root(), tree.nil(), pre);
96             printTraversal("Preorder", pre);
97         }
98         else if (query == "INORDER") {
99             inorder(tree.root(), tree.nil(), in);
100             printTraversal("Inorder", in);
101         }
102         else if (query == "POSTORDER") {
103             postorder(tree.root(), tree.nil(), post);
104             printTraversal("Postorder", post);
105         }
106         else if (query == "ALL") {
107             preorder(tree.root(), tree.nil(), pre);
108             inorder(tree.root(), tree.nil(), in);
109             postorder(tree.root(), tree.nil(), post);
110
111             printTraversal("Preorder", pre);
```

```

112         printTraversal("Inorder", in);
113         printTraversal("Postorder", post);
114     }
115 }
116
117     return 0;
118 }

```

B. Output Program

```

PS D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\PR
e-RyuuZev> g++ prob5.cpp RedBlackTree.cpp -o prob5 ; ./prob5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5 Screenshot Output Program Soal 5

C. Pembahasan

Kode ini adalah implementasi Tree Traversal pada Red Black Tree (**RBTree**). Input yang diberikan berupa sejumlah bilangan bulat yang akan dimasukkan ke dalam RBTree secara berurutan, kemudian diberikan beberapa query traversal. Query yang tersedia adalah PREORDER, INORDER, POSTORDER, dan ALL. Output yang dihasilkan adalah urutan node sesuai jenis traversal yang diminta. Jika terdapat data duplikat, data tersebut tidak dimasukkan kembali ke dalam tree.

Pada program ini digunakan class `RedBlackTree` yang berisi implementasi struktur data Red Black Tree. Selain itu terdapat struct `Node` yang menyimpan nilai node, warna node, pointer ke child kiri dan kanan, pointer parent, serta penanda node sentinel (`isNil`). Node sentinel digunakan sebagai pengganti nilai `NULL` sehingga proses balancing pada RBTree menjadi lebih mudah ditangani.

Pertama, program membaca jumlah data yang akan dimasukkan ke dalam tree. Setelah itu setiap data dibaca satu per satu. Sebelum melakukan insert, program memanggil fungsi `contains()` untuk memeriksa apakah nilai tersebut sudah ada di dalam tree. Jika nilai belum ada, maka fungsi `insert()` dipanggil untuk memasukkan node baru ke dalam RBTree. Dengan cara ini, data duplikat dapat diabaikan.

Pada fungsi `insert()`, node baru pertama kali dimasukkan seperti pada Binary Search Tree (BST). Program melakukan penelusuran dari root hingga menemukan posisi yang sesuai berdasarkan perbandingan nilai.

Jika nilai lebih kecil dari node saat ini maka bergerak ke subtree kiri, sedangkan jika lebih besar bergerak ke subtree kanan. Setelah posisi ditemukan, node baru dimasukkan sebagai leaf node berwarna merah. Selanjutnya fungsi `fixInsert()` dipanggil untuk menjaga seluruh aturan Red Black Tree tetap terpenuhi.

Fungsi `fixInsert()` bertugas melakukan proses balancing setelah insert. Jika ditemukan dua node merah yang bersebelahan, program akan melakukan recoloring atau rotasi menggunakan fungsi `rotateLeft()` dan `rotateRight()`. Proses ini dilakukan sampai seluruh aturan Red Black Tree kembali valid dan root dipastikan berwarna hitam. Dengan adanya proses balancing ini, tinggi tree tetap terjaga sehingga operasi pencarian dan penyisipan dapat dilakukan secara efisien.

Setelah seluruh data berhasil dimasukkan, program membaca jumlah query yang akan diproses. Untuk setiap query, program akan memanggil fungsi traversal yang sesuai. Traversal dilakukan menggunakan fungsi rekursif yaitu `preorder()`, `inorder()`, dan `postorder()`. Hasil traversal disimpan terlebih dahulu ke dalam vector sebelum dicetak menggunakan fungsi `printTraversal()`.

Fungsi `preorder` bekerja dengan mengunjungi root terlebih dahulu, kemudian subtree kiri, lalu subtree kanan. Fungsi `inorder` bekerja dengan mengunjungi subtree kiri, kemudian root, lalu subtree kanan. Sedangkan fungsi `postorder` mengunjungi subtree kiri, subtree kanan, kemudian root. Jika query yang diberikan adalah ALL, maka ketiga traversal dijalankan secara berurutan dan seluruh hasilnya dicetak ke layar.

Untuk kompleksitas waktu, proses pencarian duplikat menggunakan fungsi `contains()` membutuhkan $O(\log N)$ karena tinggi RBTREE dijaga tetap seimbang. Proses insert juga membutuhkan $O(\log N)$, termasuk proses balancing yang dilakukan setelah penyisipan node. Karena proses tersebut dilakukan sebanyak N kali, maka kompleksitas pembentukan tree adalah $O(N \log N)$.

Pada proses traversal, setiap node dikunjungi tepat satu kali. Oleh karena itu kompleksitas waktu untuk `preorder`, `inorder`, maupun `postorder` adalah $O(N)$.

Dalam notasi Big-O, konstanta pengali diabaikan karena yang diperhatikan adalah laju pertumbuhan ketika N sangat besar. Jika query yang diberikan adalah ALL, maka ketiga traversal dijalankan sehingga kompleksitasnya menjadi $O(3N)$, yang dapat disederhanakan menjadi $O(N)$.

Secara keseluruhan, kompleksitas waktu program adalah:

$$\text{Time Complexity} = O(N \log N + Q \times N)$$

dengan N sebagai jumlah node dalam tree dan Q sebagai jumlah query traversal yang diberikan.

Untuk kompleksitas ruang, RBTREE menyimpan seluruh node yang dimasukkan sehingga membutuhkan memori sebesar $O(N)$. Selain itu terdapat vector yang digunakan untuk menyimpan hasil traversal sebelum dicetak. Pada kondisi terburuk vector juga dapat menyimpan seluruh node tree. Oleh karena itu kompleksitas ruang program adalah:

$$\text{Space Complexity} = O(N).$$

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-RyuuZev>